

Linguagens Formais e Autômatos

MiniLang / MiniLexer

Relatório Técnico

Analisador léxico com Expressões Regulares e AFN ϵ

Professor: Daniel Leal Souza

Integrantes:

- Andrey Lourival Andrade Garcia
- Igor Cecim Vilhena

Belém – PA

30 de setembro de 2026

Resumo

O presente relatório descreve o desenvolvimento do **MiniLang** / **MiniLexer**, uma aplicação de análise léxica construída para relacionar Expressões Regulares, autômatos finitos não determinísticos com transições vazias (AFN_ϵ) e uma implementação computacional funcional. O projeto define seis Expressões Regulares principais para identificadores, inteiros, decimais, notação científica, strings e horários. Para cada expressão foram especificados o alfabeto, a linguagem, a forma formal, a sintaxe em Python, o AFN_ϵ de Thompson e casos de teste. A implementação inclui um simulador genérico de AFN_ϵ e um lexer com prioridade lexical, maior lexema, tratamento de comentários, palavras reservadas, operadores, delimitadores e erros com linha e coluna. Na execução final, a suíte automatizada apresentou 129 testes aprovados. As validações cruzadas $\text{AFN}_\epsilon \times \text{regex}$ não apresentaram divergências nos espaços de busca reduzidos definidos para as seis ERs, e a validação estrutural confirmou 6/6 AFN_ϵ em Python e 6/6 arquivos JFLAP contra as tabelas canônicas.

Palavras-chave: Expressões Regulares; AFN_ϵ ; Thompson; análise léxica; MiniLang; MiniLexer.

Sumário

1	Introdução e objetivos	1
1.1	Contexto	1
1.2	Objetivo geral	1
1.3	Objetivos específicos	1
1.4	Escopo	2
2	Fundamentação e metodologia	3
2.1	Linguagens regulares e Expressões Regulares	3
2.2	Construção de Thompson	3
2.3	Simulação de AFN_ϵ	3
2.4	Critério de validação	4
3	Especificação da MiniLang	5
3.1	Vocabulário geral	5
3.2	Palavras reservadas	5
3.3	Tokens principais	5
3.4	Seis Expressões Regulares principais	5
3.5	Ordem de tokenização	6
3.6	Regras auxiliares importantes	7
3.7	ER-01 — Identificador estruturado	7
3.7.1	Finalidade	7
3.7.2	Alfabeto	7
3.7.3	Linguagem reconhecida	7
3.7.4	ER formal	7
3.7.5	Sintaxe implementada em Python	7
3.7.6	Operadores utilizados	7
3.7.7	Equivalência formal e computacional	8
3.7.8	AFN_ϵ -epsilon	8
3.7.9	Testes dirigidos	8
3.7.10	Casos-limite	8

3.7.11	Observação sobre palavras reservadas	8
3.8	ER-02 — Inteiro sem zero à esquerda	9
3.8.1	Finalidade	9
3.8.2	Alfabeto	9
3.8.3	Linguagem reconhecida	9
3.8.4	ER formal	9
3.8.5	Sintaxe implementada em Python	9
3.8.6	Operadores utilizados	9
3.8.7	Equivalência formal e computacional	9
3.8.8	AFN-epsilon	10
3.8.9	Testes dirigidos	10
3.8.10	Casos-limite	10
3.9	ER-03 — Decimal	10
3.9.1	Finalidade	10
3.9.2	Alfabeto	10
3.9.3	Linguagem reconhecida	11
3.9.4	ER formal	11
3.9.5	Sintaxe implementada em Python	11
3.9.6	Operadores utilizados	11
3.9.7	Equivalência formal e computacional	11
3.9.8	AFN-epsilon	11
3.9.9	Testes dirigidos	12
3.9.10	Casos-limite	12
3.10	ER-04 — Número científico	12
3.10.1	Finalidade	12
3.10.2	Alfabeto	12
3.10.3	Linguagem reconhecida	12
3.10.4	ER formal	12
3.10.5	Sintaxe implementada em Python	13
3.10.6	Operadores utilizados	13
3.10.7	Equivalência formal e computacional	13
3.10.8	AFN-epsilon	13
3.10.9	Testes dirigidos	13
3.10.10	Casos-limite	14
3.11	ER-05 — Literal de string	14
3.11.1	Finalidade	14
3.11.2	Alfabeto	14
3.11.3	Linguagem reconhecida	14

3.11.4	ER formal	14
3.11.5	Sintaxe implementada em Python	14
3.11.6	Operadores utilizados	14
3.11.7	Equivalência formal e computacional	15
3.11.8	AFN-epsilon	15
3.11.9	Testes dirigidos	15
3.11.10	Casos-limite	15
3.11.11	Limitação específica	15
3.12	ER-06 — Literal de horário	16
3.12.1	Finalidade	16
3.12.2	Alfabeto	16
3.12.3	Linguagem reconhecida	16
3.12.4	ER formal	16
3.12.5	Sintaxe implementada em Python	16
3.12.6	Operadores utilizados	16
3.12.7	Equivalência formal e computacional	16
3.12.8	AFN-epsilon	17
3.12.9	Testes dirigidos	17
3.12.10	Casos-limite	17
3.12.11	Observação de correção	17
4	Implementação do MiniLexer	18
4.1	Arquitetura do código	18
4.2	Processamento léxico	18
4.3	Proteção contra fragmentação	18
4.4	Localização dos erros	19
4.5	Interface de execução	19
4.6	Exemplos fornecidos	19
5	Testes e validação	20
5.1	Suíte de testes	20
5.2	Validação cruzada $AFN_{\epsilon} \times regex$	20
5.3	Validação estrutural dos AFN_{ϵ}	20
5.4	Interpretação dos resultados	21
5.5	Pontos de competição no lexer	21
6	Limitações e discussão	22
6.1	Limitações da linguagem	22
6.2	Limitações do lexer	22

6.3	Melhorias futuras	22
6.4	Decisões de escopo	23
7	Documentação, reprodutibilidade e uso de IA	24
7.1	Organização do projeto	24
7.2	Reprodutibilidade	24
7.3	Uso de Inteligência Artificial	24
7.4	Registro de contribuições	25
8	Conclusão	26
9	Mapeamento dos artefatos	27
A	Tabelas completas dos AFN-epsilon	28
A.1	ER-01 — Identificador estruturado	28
A.2	ER-02 — Inteiro sem zero à esquerda	29
A.3	ER-03 — Decimal	29
A.4	ER-04 — Número científico	30
A.5	ER-05 — Literal de string	32
A.6	ER-06 — Literal de horário	32
	Referências	33

Capítulo 1

Introdução e objetivos

1.1 Contexto

O projeto **MiniLang / MiniLexer** foi desenvolvido para a disciplina de Linguagens Formais e Autômatos com o objetivo de relacionar os conceitos de Expressões Regulares e autômatos finitos com uma aplicação computacional funcional. A aplicação implementa um analisador léxico de uma linguagem de programação simplificada denominada MiniLang.

O enunciado da atividade exige uma aplicação com entrada, processamento e saída, no mínimo cinco Expressões Regulares relevantes, tratamento de entradas inválidas, código organizado e, para cada expressão, a apresentação do alfabeto, linguagem reconhecida, ER formal, sintaxe implementada, explicação dos operadores, AFN ϵ e testes com cadeias aceitas e rejeitadas. Também estabelece que a ER formal, a sintaxe usada no programa, os testes e o AFN ϵ devem representar a mesma linguagem.

1.2 Objetivo geral

Construir e validar um analisador léxico da MiniLang, demonstrando uma cadeia completa de consistência:

especificação $\rightarrow$ ER formal $\rightarrow$ AFN ϵ $\rightarrow$ regex Python $\rightarrow$ testes $\rightarrow$ lexer

1.3 Objetivos específicos

- definir seis Expressões Regulares próprias e relevantes para a MiniLang;
- construir os AFN ϵ correspondentes pela convenção de Thompson adotada no projeto;

- implementar um simulador genérico de $\text{AFN}\varepsilon$;
- implementar o MiniLexer com prioridade e maior lexema;
- validar a correspondência entre $\text{AFN}\varepsilon$ e regex Python por testes automatizados;
- validar estruturalmente estados, estados finais, alfabetos e transições dos seis $\text{AFN}\varepsilon$;
- disponibilizar os diagramas em JFLAP e Mermaid;
- documentar limitações, resultados e possibilidades de evolução.

1.4 Escopo

A MiniLang é tratada exclusivamente no nível léxico. O projeto não implementa parser, análise semântica, compilação, execução, verificação completa de tipos, geração de código ou máquina virtual. Esta delimitação evita que funcionalidades posteriores alterem o foco acadêmico do trabalho.

Capítulo 2

Fundamentação e metodologia

2.1 Linguagens regulares e Expressões Regulares

As Expressões Regulares são utilizadas para descrever subconjuntos de cadeias sobre alfabetos finitos. No projeto, a notação formal emprega união, concatenação, fecho de Kleene, fecho positivo, opcionalidade, agrupamento e classes finitas. A sintaxe Python corresponde a essa descrição por meio do módulo `re`.

A validação em Python utiliza `re.fullmatch()`, de modo que a cadeia inteira seja comparada ao padrão. Isso evita que um prefixo válido seja confundido com uma cadeia integralmente válida.

2.2 Construção de Thompson

Todos os seis AFN_ε usam a mesma convenção de Thompson definida no projeto: um átomo gera dois estados e uma transição rotulada; concatenações são ligadas por transições ε ; uniões usam novos estados inicial e final com transições ε ; e o fecho de Kleene usa novas ligações ε para entrada, saída e retorno.

Classes finitas são apresentadas como rótulos agrupados nos diagramas, mas são expandidas para símbolos individuais no simulador Python. Assim, por exemplo, D representa o conjunto $\{0, 1, \dots, 9\}$.

2.3 Simulação de AFN_ε

O simulador inicia no ε -fecho do estado inicial. Para cada símbolo da cadeia, aplica a operação *move* sobre os estados ativos e, em seguida, calcula novamente o ε -fecho. Ao fim da cadeia, a aceitação ocorre quando pelo menos um estado ativo pertence ao conjunto de estados finais.

A implementação utiliza `EPSILON =` apenas como marcador interno de transições vazias. A cadeia vazia continua sendo uma entrada distinta do símbolo lógico ε .

2.4 Critério de validação

A validação foi organizada em três níveis:

1. **casos dirigidos:** pelo menos seis cadeias aceitas e seis rejeitadas por ER, incluindo fronteiras;
2. **validação cruzada:** comparação entre a decisão do AFN_ε e `re.fullmatch()`;
3. **validação estrutural:** comparação exata entre tabelas canônicas, implementação Python e arquivos JFLAP.

Essa separação evita que uma eventual construção incorreta passe despercebida apenas porque o AFN e a regex foram derivados do mesmo erro. A validação estrutural também impede que dois autômatos diferentes, mas semanticamente equivalentes, sejam tratados como a mesma construção.

Capítulo 3

Especificação da MiniLang

3.1 Vocabulário geral

O alfabeto geral da MiniLang utiliza letras ASCII, dígitos, sublinhado, operadores, delimitadores, ponto decimal, aspas e espaços de acordo com o papel de cada categoria lexical. Palavras reservadas são reconhecidas por uma tabela auxiliar e não constituem uma das seis ERs principais.

3.2 Palavras reservadas

A tabela de palavras reservadas contém:

```
program, int, float, string, bool, time,  
if, else, while, print, true, false.
```

3.3 Tokens principais

Os tokens definidos pelo projeto incluem `PROGRAM`, tipos, estruturas de controle, `BOOLEAN`, identificadores, inteiros, decimais, notação científica, strings, horários, operadores e delimitadores. `WHITESPACE` e `COMMENT` são classes internas e são descartadas da saída final.

3.4 Seis Expressões Regulares principais

As seis ERs principais são:

ER	Estados	Transições	ε	Rotuladas
ER-01	28	35	27	8
ER-02	10	12	9	3
ER-03	18	22	16	6
ER-04	42	52	40	12
ER-05	8	9	6	3
ER-06	16	16	9	7

As contagens acima são as contagens definitivas congeladas no projeto.

3.5 Ordem de tokenização

A ordem oficial é:

1. whitespace;
2. comentário;
3. string;
4. operador composto;
5. número científico;
6. número decimal;
7. horário;
8. número inteiro;
9. identificador ou palavra reservada;
10. operador simples;
11. delimitador;
12. erro léxico.

A prioridade é combinada com o princípio de maior lexema. Em especial, o lexer não deve transformar um candidato numérico inválido em vários tokens menores.

3.6 Regras auxiliares importantes

Comentários iniciados por `//` são ignorados antes que `/` seja tratado como divisão. Operadores compostos `==`, `!=`, `<=` e `>=` têm prioridade sobre operadores simples. A string é analisada antes das regras internas que poderiam interpretar seus símbolos como operadores. O ponto isolado não é um token válido.

3.7 ER-01 — Identificador estruturado

3.7.1 Finalidade

Reconhecer identificadores válidos da MiniLang.

3.7.2 Alfabeto

$\Sigma_{ID} = L \cup D \cup \{_ \}$, com L para letras e D para dígitos.

3.7.3 Linguagem reconhecida

A cadeia começa obrigatoriamente com uma letra; depois disso podem ocorrer caracteres alfanuméricos ou blocos iniciados por sublinhado, sendo que cada sublinhado deve ser seguido por pelo menos um caractere alfanumérico.

3.7.4 ER formal

$L(A \mid _AA^*)^*$, com $A = L \cup D$.

3.7.5 Sintaxe implementada em Python

```
r"[A-Za-z]([A-Za-z0-9]|\_[A-Za-z0-9]+)*"
```

3.7.6 Operadores utilizados

A união `|` permite escolher entre alternativas; a concatenação liga blocos em sequência; o fecho de Kleene `*` permite zero ou mais repetições; e o fecho positivo `+` indica uma ou mais ocorrências no ramo `\_[A-Za-z0-9]+`. Os parênteses agrupam subexpressões e as classes `[A-Za-z]` e `[A-Za-z0-9]` representam conjuntos finitos de símbolos.

3.7.7 Equivalência formal e computacional

Formal	Python
L	<code>[A-Za-z]</code>
$A = L \cup D$	<code>[A-Za-z0-9]</code>
$_AA^*$	<code>_[A-Za-z0-9]+</code>
$+ \text{ como } AA^*$	<code>+</code>

3.7.8 AFN-epsilon

Estado inicial: q0. Estado final: q27. A construção possui 28 estados, 35 transições, 27 transições ε e 8 transições rotuladas. Os diagramas JFLAP e Mermaid correspondentes estão em docs/diagramas/ER-01/. Ambos são derivados da mesma tabela canônica.

3.7.9 Testes dirigidos

Cadeia	Esperado	Interpretação
a	Aceita	Pertence à linguagem definida.
nome	Aceita	Pertence à linguagem definida.
aluno1	Aceita	Pertence à linguagem definida.
valor_total	Aceita	Pertence à linguagem definida.
mediaFinal2	Aceita	Pertence à linguagem definida.
a1_b2	Aceita	Pertence à linguagem definida.
ε	Rejeita	Não pertence à linguagem definida.
1aluno	Rejeita	Não pertence à linguagem definida.
_nome	Rejeita	Não pertence à linguagem definida.
nome_	Rejeita	Não pertence à linguagem definida.
nome__aluno	Rejeita	Não pertence à linguagem definida.
nome-aluno	Rejeita	Não pertence à linguagem definida.

3.7.10 Casos-limite

Os casos de fronteira selecionados são: a, ε , a_, a_b. O propósito desses casos é explorar as menores cadeias válidas, limites superiores/inferiores e formas quase válidas que poderiam ser aceitas por uma regra permissiva demais.

3.7.11 Observação sobre palavras reservadas

Palavras como `int` e `float` possuem formato compatível com a ER de identificadores. A distinção final ocorre pela tabela de palavras reservadas do lexer, após o reconhecimento do lexema.

3.8 ER-02 — Inteiro sem zero à esquerda

3.8.1 Finalidade

Reconhecer inteiros sem zero à esquerda, permitindo somente o literal ‘0’ como zero isolado.

3.8.2 Alfabeto

$\Sigma_{INT} = D$, com $D = \{0, 1, \dots, 9\}$.

3.8.3 Linguagem reconhecida

$L_{INT} = \{0\} \cup ND^*$, onde $N = \{1, \dots, 9\}$.

3.8.4 ER formal

$0 \mid ND^*$.

3.8.5 Sintaxe implementada em Python

```
r"(0|[1-9][0-9]*)"
```

3.8.6 Operadores utilizados

A união $\mid$ separa o caso do zero do caso iniciado por N ; a concatenação forma a sequência dos símbolos; o fecho $*$ permite qualquer quantidade de dígitos após o primeiro não nulo; e os parênteses agrupam a alternativa no padrão Python.

3.8.7 Equivalência formal e computacional

Formal	Python
0	0
N	[1-9]
D	[0-9]
D^*	[0-9]*

3.8.8 AFN-epsilon

Estado inicial: q0. Estado final: q1. A construção possui 10 estados, 12 transições, 9 transições ε e 3 transições rotuladas. Os diagramas JFLAP e Mermaid correspondentes estão em docs/diagramas/ER-02/. Ambos são derivados da mesma tabela canônica.

3.8.9 Testes dirigidos

Cadeia	Esperado	Interpretação
0	Aceita	Pertence à linguagem definida.
1	Aceita	Pertence à linguagem definida.
7	Aceita	Pertence à linguagem definida.
10	Aceita	Pertence à linguagem definida.
2026	Aceita	Pertence à linguagem definida.
999999	Aceita	Pertence à linguagem definida.
ε	Rejeita	Não pertence à linguagem definida.
00	Rejeita	Não pertence à linguagem definida.
01	Rejeita	Não pertence à linguagem definida.
007	Rejeita	Não pertence à linguagem definida.
12.5	Rejeita	Não pertence à linguagem definida.
-10	Rejeita	Não pertence à linguagem definida.

3.8.10 Casos-limite

Os casos de fronteira selecionados são: 0, 1, 00, 01, ε . O propósito desses casos é explorar as menores cadeias válidas, limites superiores/inferiores e formas quase válidas que poderiam ser aceitas por uma regra permissiva demais.

3.9 ER-03 — Decimal

3.9.1 Finalidade

Reconhecer números decimais cuja parte inteira segue a regra de inteiro e cuja parte fracionária possui ao menos um dígito.

3.9.2 Alfabeto

$$\Sigma_{DEC} = D \cup \{.\}.$$

3.9.3 Linguagem reconhecida

A cadeia possui uma parte inteira válida, seguida por ponto e por uma ou mais ocorrências de dígitos.

3.9.4 ER formal

$(0 \mid ND^*)PDD^*$, com $P = \{.\}$.

3.9.5 Sintaxe implementada em Python

```
r"(0|[1-9][0-9]*)\.[0-9]+"
```

3.9.6 Operadores utilizados

A união $\mid$ define as alternativas da parte inteira; a concatenação coloca parte inteira, ponto e parte fracionária em sequência; o ponto é tratado como literal por meio de $P = \{.\}$ e do escape $\backslash$. no código; e o $+$ computacional exige pelo menos um dígito após o ponto.

3.9.7 Equivalência formal e computacional

Formal	Python
N	[1-9]
D	[0-9]
P	$\backslash$.
DD^*	[0-9] +

3.9.8 AFN-epsilon

Estado inicial: q0. Estado final: q17. A construção possui 18 estados, 22 transições, 16 transições ε e 6 transições rotuladas. Os diagramas JFLAP e Mermaid correspondentes estão em docs/diagramas/ER-03/. Ambos são derivados da mesma tabela canônica.

3.9.9 Testes dirigidos

Cadeia	Esperado	Interpretação
0.5	Aceita	Pertence à linguagem definida.
0.50	Aceita	Pertence à linguagem definida.
1.5	Aceita	Pertence à linguagem definida.
9.5	Aceita	Pertence à linguagem definida.
10.25	Aceita	Pertence à linguagem definida.
2026.75	Aceita	Pertence à linguagem definida.
00.5	Rejeita	Não pertence à linguagem definida.
09.5	Rejeita	Não pertence à linguagem definida.
.5	Rejeita	Não pertence à linguagem definida.
1.	Rejeita	Não pertence à linguagem definida.
1.5.2	Rejeita	Não pertence à linguagem definida.
-10.5	Rejeita	Não pertence à linguagem definida.

3.9.10 Casos-limite

Os casos de fronteira selecionados são: 0.0, 1.0, 00.5, .5, 1.. O propósito desses casos é explorar as menores cadeias válidas, limites superiores/inferiores e formas quase válidas que poderiam ser aceitas por uma regra permissiva demais.

3.10 ER-04 — Número científico

3.10.1 Finalidade

Reconhecer números em notação científica com parte inteira válida, parte decimal opcional, expoente iniciado por ‘e’ ou ‘E’, sinal opcional e expoente numérico obrigatório.

3.10.2 Alfabeto

$$\Sigma_{SCI} = D \cup \{., e, E, +, -\}.$$

3.10.3 Linguagem reconhecida

A base é 0 ou ND^* ; a parte decimal é opcional; a letra e ou E é obrigatória; o sinal do expoente é opcional; o expoente possui um ou mais dígitos.

3.10.4 ER formal

$$(0 \mid ND^*)(PDD^* \mid \varepsilon)(e \mid E)(+ \mid - \mid \varepsilon)DD^*.$$

3.10.5 Sintaxe implementada em Python

```
r"(0|[1-9][0-9]*) (\.[0-9]+)? [eE] [+]? [0-9]+"
```

3.10.6 Operadores utilizados

A união $|$ representa alternativas; a concatenação encadeia a mantissa, o expoente e seus componentes; a opcionalidade $?$ corresponde a $r | \varepsilon$ para a parte decimal; as classes $[eE]$ e $[+-]$ representam escolhas finitas; e $[0-9]^+$ exige um ou mais dígitos no expoente.

3.10.7 Equivalência formal e computacional

Formal	Python
$PDD^* \varepsilon$	$(\backslash.[0-9]^+)?$
$e E$	$[eE]$
$+ - \varepsilon$	$[+-]?$
DD^*	$[0-9]^+$

3.10.8 AFN-epsilon

Estado inicial: q0. Estado final: q39. A construção possui 42 estados, 52 transições, 40 transições ε e 12 transições rotuladas. Os diagramas JFLAP e Mermaid correspondentes estão em docs/diagramas/ER-04/. Ambos são derivados da mesma tabela canônica.

3.10.9 Testes dirigidos

Cadeia	Esperado	Interpretação
1e3	Aceita	Pertence à linguagem definida.
10e2	Aceita	Pertence à linguagem definida.
0e5	Aceita	Pertence à linguagem definida.
1.5e3	Aceita	Pertence à linguagem definida.
9.5E-2	Aceita	Pertence à linguagem definida.
10.25e+4	Aceita	Pertence à linguagem definida.
ε	Rejeita	Não pertence à linguagem definida.
1e	Rejeita	Não pertence à linguagem definida.
1e+	Rejeita	Não pertence à linguagem definida.
.5e2	Rejeita	Não pertence à linguagem definida.
01e3	Rejeita	Não pertence à linguagem definida.
1.5.2e3	Rejeita	Não pertence à linguagem definida.

3.10.10 Casos-limite

Os casos de fronteira selecionados são: 1e0, 0e1, 1e, 1e+, 01e3. O propósito desses casos é explorar as menores cadeias válidas, limites superiores/inferiores e formas quase válidas que poderiam ser aceitas por uma regra permissiva demais.

3.11 ER-05 — Literal de string

3.11.1 Finalidade

Reconhecer literais de string delimitados por aspas duplas, com conteúdo em um conjunto ASCII restrito.

3.11.2 Alfabeto

$C = L \cup D \cup \{\text{espaço}, _, ., ,, !, ?, :, ;, +, -, *, /, =, <, >\}$ e $\Sigma_{STR} = C \cup \{\text{aspas duplas}\}$.

3.11.3 Linguagem reconhecida

$L_{STR} = \{\text{aspas duplas } x \text{ aspas duplas} \mid x \in C^*\}$. A string vazia é válida.

3.11.4 ER formal

aspas duplas C^* aspas duplas.

3.11.5 Sintaxe implementada em Python

```
r'"[A-Za-z0-9 _.,!?:;+*/=<>-]*"'
```

3.11.6 Operadores utilizados

A classe finita do conteúdo representa os símbolos permitidos; o fecho de Kleene $*$ permite zero ou mais caracteres dentro das aspas; e as aspas duplas são delimitadores literais da cadeia. Não há mecanismo de escape nesta versão.

3.11.7 Equivalência formal e computacional

Formal	Python
C^*	<code>[A-Za-z0-9 _.,! ? ; : + * / = < > -] *</code>
aspas	<code>" . . . "</code>
* permite ε	<code>*</code>

3.11.8 AFN-epsilon

Estado inicial: q0. Estado final: q7. A construção possui 8 estados, 9 transições, 6 transições ε e 3 transições rotuladas. Os diagramas JFLAP e Mermaid correspondentes estão em docs/diagramas/ER-05/. Ambos são derivados da mesma tabela canônica.

3.11.9 Testes dirigidos

Cadeia	Esperado	Interpretação
"Andrey"	Aceita	Pertence à linguagem definida.
"abc 123"	Aceita	Pertence à linguagem definida.
"a_b.c"	Aceita	Pertence à linguagem definida.
"hello!"	Aceita	Pertence à linguagem definida.
"12:30"	Aceita	Pertence à linguagem definida.
ε	Rejeita	Não pertence à linguagem definida.
"abc	Rejeita	Não pertence à linguagem definida.
abc"	Rejeita	Não pertence à linguagem definida.
"a"b"	Rejeita	Não pertence à linguagem definida.
"a\{ }b"	Rejeita	Não pertence à linguagem definida.
"linha	Rejeita	Não pertence à linguagem definida.

3.11.10 Casos-limite

Os casos de fronteira selecionados são: `"a"`, `"abc"`, `"a\{ }b"`. O propósito desses casos é explorar as menores cadeias válidas, limites superiores/inferiores e formas quase válidas que poderiam ser aceitas por uma regra permissiva demais.

3.11.11 Limitação específica

A primeira versão da MiniLang não possui mecanismo de escape. Aspas, barra invertida, tabulação e quebra de linha não pertencem ao conjunto de conteúdo C .

3.12 ER-06 — Literal de horário

3.12.1 Finalidade

Reconhecer literais de horário no formato HH:MM, com horas entre 00 e 23 e minutos entre 00 e 59.

3.12.2 Alfabeto

$$\Sigma_{TIME} = D \cup \{:\}.$$

3.12.3 Linguagem reconhecida

A hora pertence a 00–19 ou 20–23; os minutos pertencem a 00–59.

3.12.4 ER formal

$$((0 \mid 1)D \mid 2(0 \mid 1 \mid 2 \mid 3))Q(0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5)D, \text{ com } Q = \{:\}.$$

3.12.5 Sintaxe implementada em Python

```
r"([01][0-9]|2[0-3]):[0-5][0-9]"
```

3.12.6 Operadores utilizados

A união $\mid$ define as alternativas para as horas e minutos; a concatenação organiza os dígitos e o separador; as classes $[01]$, $[0-3]$ e $[0-5]$ representam os conjuntos finitos correspondentes.

3.12.7 Equivalência formal e computacional

Formal	Python
$(0 \mid 1)$	<code>[01]</code>
D	<code>[0-9]</code>
$2(0 \mid 1 \mid 2 \mid 3)$	<code>2[0-3]</code>
Q	<code>:</code>
$(0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5)$	<code>[0-5]</code>

3.12.8 AFN-epsilon

Estado inicial: q0. Estado final: q15. A construção possui 16 estados, 16 transições, 9 transições ε e 7 transições rotuladas. Os diagramas JFLAP e Mermaid correspondentes estão em docs/diagramas/ER-06/. Ambos são derivados da mesma tabela canônica.

3.12.9 Testes dirigidos

Cadeia	Esperado	Interpretação
00:00	Aceita	Pertence à linguagem definida.
09:05	Aceita	Pertence à linguagem definida.
12:30	Aceita	Pertence à linguagem definida.
14:59	Aceita	Pertence à linguagem definida.
23:00	Aceita	Pertence à linguagem definida.
23:59	Aceita	Pertence à linguagem definida.
ε	Rejeita	Não pertence à linguagem definida.
24:00	Rejeita	Não pertence à linguagem definida.
12:60	Rejeita	Não pertence à linguagem definida.
2:30	Rejeita	Não pertence à linguagem definida.
14:5	Rejeita	Não pertence à linguagem definida.
99:99	Rejeita	Não pertence à linguagem definida.

3.12.10 Casos-limite

Os casos de fronteira selecionados são: 00:00, 23:59, 24:00, 12:60. O propósito desses casos é explorar as menores cadeias válidas, limites superiores/inferiores e formas quase válidas que poderiam ser aceitas por uma regra permissiva demais.

3.12.11 Observação de correção

Na faixa de horas 20–23, o segundo dígito pertence ao conjunto $\{0, 1, 2, 3\}$.

Nos arquivos JFLAP e Mermaid, essa transição aparece como [0-3].

Capítulo 4

Implementação do MiniLexer

4.1 Arquitetura do código

A implementação foi organizada em módulos:

- `src/automata.py`: marcador de ε , classe `AFNe`, simulador e construções dos seis `AFNe`;
- `src/patterns.py`: regexes de referência, palavras reservadas, operadores, delimitadores e conjuntos auxiliares;
- `src/tokens.py`: tipos e representação de tokens;
- `src/lexer.py`: análise léxica, controle de posição, maior lexema e erros;
- `src/main.py`: interface de linha de comando.

4.2 Processamento léxico

O lexer percorre a entrada da esquerda para a direita. Espaços são descartados; comentários de linha iniciados por `//` são ignorados; strings são consumidas integralmente entre aspas; operadores compostos são reconhecidos antes dos simples; números são analisados como candidatos que podem ser inteiros, decimais, científicos ou horários; identificadores são reconhecidos e posteriormente classificados como palavras reservadas ou `IDENTIFIER`.

4.3 Proteção contra fragmentação

Um dos pontos relevantes da implementação é evitar que uma entrada inválida seja silenciosamente dividida em tokens menores. Exemplos como `007`, `09.5`, `1.5e`, `24:00` e `10a`

são tratados como candidatos inválidos e geram erro léxico, em vez de serem decompostos em sequências aparentemente válidas.

4.4 Localização dos erros

A classe `LexicalError` registra linha, coluna, lexema e motivo. O lexer atualiza essas posições ao consumir caracteres, preservando informação útil inclusive após linhas vazias, tabulações e terminações de linha CRLF.

4.5 Interface de execução

A aplicação pode receber um arquivo `.min` pela linha de comando ou solicitar uma linha de código quando executada sem arquivo. A entrada vazia recebe mensagem controlada. Em caso de erro léxico, a execução informa posição, lexema e motivo.

4.6 Exemplos fornecidos

O arquivo `examples/valido.min` demonstra declarações, números, string, booleano, horário, condicional, repetição, impressão e comentário. O arquivo `examples/invalido.min` reúne casos deliberadamente inválidos, como zero à esquerda, decimal com zero à esquerda, notação científica sem expoente completo, horário inválido, string sem fechamento e símbolo desconhecido.

Capítulo 5

Testes e validação

5.1 Suíte de testes

A suíte atual possui **129 testes coletados**, todos aprovados na execução final com `pytest -q`.

Os testes cobrem palavras reservadas, identificadores, números, horários, strings, comentários, operadores, maior lexema, erros léxicos, linha/coluna, arquivos de exemplo e execução da interface.

5.2 Validação cruzada $\text{AFN}_\varepsilon \times \text{regex}$

Além da suíte de integração, cada uma das seis ERs possui um validator individual e um validator geral. A validação compara a aceitação do AFN_ε com a aplicação de `re.fullmatch()` sobre alfabetos reduzidos.

ER	Cadeias comparadas	Divergências
ER-01	3.906	0
ER-02	111.111	0
ER-03	364	0
ER-04	19.608	0
ER-05	19.531	0
ER-06	177.156	0

5.3 Validação estrutural dos AFN_ε

A validação estrutural compara a especificação canônica com a implementação Python e com os seis arquivos JFLAP. São verificadas: conjunto de estados, estado inicial, estados finais, alfabeto, todas as transições e contagens de Thompson. Para JFLAP, também são verificados os rótulos explícitos das classes agrupadas.

ER	Python	JFLAP
ER-01	OK	OK
ER-02	OK	OK
ER-03	OK	OK
ER-04	OK	OK
ER-05	OK	OK
ER-06	OK	OK

Resultado consolidado: **6/6 AFN ϵ Python + 6/6 JFLAP**, com coincidência exata das tabelas, estados, finais e transições.

5.4 Interpretação dos resultados

Os testes não constituem uma prova abstrata de equivalência para todos os alfabetos possíveis. Eles demonstram ausência de divergência no conjunto de casos dirigidos e nas buscas exaustivas sobre os alfabetos reduzidos definidos para cada ER. A validação estrutural complementa essa evidência ao verificar que a implementação e os arquivos JFLAP preservam a construção documentada.

5.5 Pontos de competição no lexer

Os testes de integração verificam explicitamente situações em que duas regras poderiam competir. Entre elas: comentário versus divisão, operadores compostos versus simples, científico versus decimal, decimal versus inteiro, palavras reservadas versus identificadores e candidatos numéricos potencialmente inválidos. Também são verificados strings contendo símbolos que, fora das aspas, teriam outro significado léxico.

Capítulo 6

Limitações e discussão

6.1 Limitações da linguagem

A MiniLang continua restrita ao nível lexical. O reconhecimento de um identificador não determina se ele foi declarado ou qual seu papel semântico. O reconhecimento de um número verifica somente seu formato textual. Horários são validados segundo as faixas 00–23 e 00–59, mas não possuem interpretação contextual.

Strings não possuem escapes na primeira versão. Isso simplifica a construção do AFN_{ϵ} , mas limita a representação de conteúdos que exijam aspas ou barra invertida internas.

6.2 Limitações do lexer

O MiniLexer não constrói árvore sintática nem interpreta a estrutura do programa. O arquivo pode ser lexicalmente válido sem ser um programa sintaticamente válido em uma linguagem de programação completa.

6.3 Melhorias futuras

Como evolução do projeto, podem ser considerados: introdução de escapes em strings com uma nova ER e correspondente AFN_{ϵ} ; expansão controlada do vocabulário da MiniLang; criação de uma camada de análise sintática; e ampliação das baterias de testes para alfabetos maiores. Essas extensões não fazem parte da versão entregue e, portanto, não foram utilizadas para produzir os resultados atuais.

6.4 Decisões de escopo

Optou-se por não acrescentar funcionalidades apenas para tornar as ERs artificialmente mais complexas. As seis ERs existentes já superam o mínimo exigido e são diretamente justificadas pelo vocabulário lexical da aplicação.

Capítulo 7

Documentação, reprodutibilidade e uso de IA

7.1 Organização do projeto

A versão atual do repositório contém código-fonte, testes, exemplos, documentação, declaração de IA, registro de contribuições e diagramas. Cada ER possui um arquivo `.jff` para JFLAP e um arquivo `.md` com Mermaid.

7.2 Reprodutibilidade

A execução dos testes é realizada com:

```
pip install -r requirements.txt
pytest -q
python tests/validate_all.py
python tests/validate_afne_structure.py
```

As classes utilizadas nos diagramas JFLAP são explicitadas, por exemplo, como `[A-Za-z]`, `[0-9]`, `[1-9]`, `[0-5]` e `[0-3]`, enquanto o simulador Python expande os conjuntos para seus símbolos constituintes.

7.3 Uso de Inteligência Artificial

Ferramentas de IA foram utilizadas como apoio na organização e revisão da especificação, análise das ERs e AFNe, implementação e revisão do simulador e do lexer, criação dos testes, conferência dos diagramas e revisão documental. As ferramentas utilizadas foram o Claude (Anthropic) e o ChatGPT (OpenAI). O projeto mantém uma declaração explícita

em `DECLARACAO_IA.md`. A responsabilidade pela compreensão, revisão e alteração do material permanece com a equipe.

7.4 Registro de contribuições

As contribuições de cada integrante estão registradas em `CONTRIBUICOES.md`:

- **Igor Cecim Vilhena:** definição e formalização das Expressões Regulares (alfabetos, linguagens, notação formal e sintaxe Python), redação do relatório técnico e elaboração dos slides da apresentação.
- **Andrey Lourival Andrade Garcia:** especificação da MiniLang, construção dos AFN ϵ de Thompson e do simulador, implementação do MiniLexer, testes automatizados e validação AFN $\epsilon \times$ regex, diagramas JFLAP e Mermaid, exemplos, README, fichas explicativas das ERs e roteiro de comandos da apresentação.

Capítulo 8

Conclusão

O MiniLang / MiniLexer atingiu uma implementação funcional coerente com o escopo definido para um analisador léxico. O projeto contém seis Expressões Regulares principais, seis construções de AFN_{ε} , simulador genérico, lexer com regras de prioridade e maior lexema, tratamento de erros e suíte de testes integrada.

A verificação final registrou 129 testes automatizados aprovados. As seis validações cruzadas $AFN_{\varepsilon} \times \text{regex Python}$ não apresentaram divergências nos alfabetos reduzidos definidos, e a validação estrutural confirmou 6/6 implementações Python e 6/6 arquivos JFLAP contra as tabelas canônicas.

A principal contribuição acadêmica da implementação está na manutenção da correspondência entre a descrição formal e o comportamento computacional. Cada ER é apresentada com sua finalidade, alfabeto, linguagem, notação formal, sintaxe Python, AFN_{ε} , casos dirigidos e limitações, enquanto os testes de integração demonstram como essas regras funcionam em conjunto no MiniLexer.

Os artefatos finais — código-fonte, testes, diagramas dos AFN_{ε} , relatório, apresentação e registro de contribuições — estão disponíveis no repositório do projeto.

Capítulo 9

Mapeamento dos artefatos

Os principais artefatos acadêmicos do repositório são:

Artefato	Finalidade
MiniLang_Guia_Projeto.md	especificação vigente da linguagem e das ERs
src/automata.py	simulador e AFN_{ϵ}
src/patterns.py	regexes e tabelas auxiliares
src/lexer.py	analisador léxico
tests/validate_erXX.py	validação individual das ERs
tests/validate_afne_	validação estrutural dos seis AFN_{ϵ}
structure.py	
docs/diagramas/ER-XX/*.jff	diagramas para JFLAP
docs/diagramas/ER-XX/ER-XX.	diagramas textuais em Mermaid
md	
DECLARACAO_IA.md	declaração de uso de IA
CONTRIBUICOES.md	registro de contribuições da equipe

Apêndice A

Tabelas completas dos AFN-epsilon

A.1 ER-01 — Identificador estruturado

No.	Origem	Símbolo	Destino
1	q0	[A-Za-z]	q1
2	q1	ε	q26
3	q6	ε	q2
4	q6	ε	q4
5	q2	[A-Za-z]	q3
6	q3	ε	q7
7	q4	[0-9]	q5
8	q5	ε	q7
9	q8	-	q9
10	q9	ε	q14
11	q14	ε	q10
12	q14	ε	q12
13	q10	[A-Za-z]	q11
14	q11	ε	q15
15	q12	[0-9]	q13
16	q13	ε	q15
17	q15	ε	q16
18	q16	ε	q18
19	q16	ε	q17
20	q18	ε	q20
21	q18	ε	q22
22	q20	[A-Za-z]	q21
23	q22	[0-9]	q23
24	q21	ε	q19

No.	Origem	Símbolo	Destino
25	q23	ε	q19
26	q19	ε	q18
27	q19	ε	q17
28	q24	ε	q6
29	q24	ε	q8
30	q7	ε	q25
31	q17	ε	q25
32	q26	ε	q24
33	q26	ε	q27
34	q25	ε	q24
35	q25	ε	q27

A.2 ER-02 — Inteiro sem zero à esquerda

No.	Origem	Símbolo	Destino
1	q0	ε	q2
2	q0	ε	q4
3	q2	0	q3
4	q3	ε	q1
5	q4	[1-9]	q5
6	q5	ε	q8
7	q8	ε	q6
8	q8	ε	q9
9	q6	[0-9]	q7
10	q7	ε	q6
11	q7	ε	q9
12	q9	ε	q1

A.3 ER-03 — Decimal

No.	Origem	Símbolo	Destino
1	q2	0	q3
2	q0	ε	q2
3	q3	ε	q1
4	q4	[1-9]	q5

No.	Origem	Símbolo	Destino
5	q8	[0-9]	q9
6	q6	ε	q8
7	q6	ε	q7
8	q9	ε	q8
9	q9	ε	q7
10	q5	ε	q6
11	q0	ε	q4
12	q7	ε	q1
13	q10	.	q11
14	q1	ε	q10
15	q12	[0-9]	q13
16	q11	ε	q12
17	q16	[0-9]	q17
18	q14	ε	q16
19	q14	ε	q17
20	q17	ε	q16
21	q17	ε	q15
22	q13	ε	q14

A.4 ER-04 — Número científico

No.	Origem	Símbolo	Destino
1	q2	0	q3
2	q0	ε	q2
3	q3	ε	q1
4	q4	[1-9]	q5
5	q8	[0-9]	q9
6	q6	ε	q8
7	q6	ε	q7
8	q9	ε	q8
9	q9	ε	q7
10	q5	ε	q6
11	q0	ε	q4
12	q7	ε	q1
13	q12	.	q13
14	q14	[0-9]	q15
15	q13	ε	q14

No.	Origem	Símbolo	Destino
16	q18	[0-9]	q19
17	q16	ε	q18
18	q16	ε	q17
19	q19	ε	q18
20	q19	ε	q17
21	q15	ε	q16
22	q10	ε	q12
23	q17	ε	q11
24	q20	ε	q21
25	q10	ε	q20
26	q21	ε	q11
27	q1	ε	q10
28	q24	e	q25
29	q22	ε	q24
30	q25	ε	q23
31	q26	E	q27
32	q22	ε	q26
33	q27	ε	q23
34	q11	ε	q22
35	q30	+	q31
36	q28	ε	q30
37	q31	ε	q29
38	q32	-	q33
39	q28	ε	q32
40	q33	ε	q29
41	q34	ε	q35
42	q28	ε	q34
43	q35	ε	q29
44	q23	ε	q28
45	q36	[0-9]	q37
46	q29	ε	q36
47	q40	[0-9]	q41
48	q38	ε	q40
49	q38	ε	q39
50	q41	ε	q40
51	q41	ε	q39
52	q37	ε	q38

A.5 ER-05 — Literal de string

No.	Origem	Símbolo	Destino
1	q0	"	q1
2	q4	[A-Za-z0-9 _ . , ! ? ; : + * / = < > -]	q5
3	q2	ε	q4
4	q2	ε	q3
5	q5	ε	q4
6	q5	ε	q3
7	q1	ε	q2
8	q6	"	q7
9	q3	ε	q6

A.6 ER-06 — Literal de horário

No.	Origem	Símbolo	Destino
1	q2	[01]	q3
2	q4	[0-9]	q5
3	q3	ε	q4
4	q0	ε	q2
5	q5	ε	q1
6	q6	2	q7
7	q8	[0-3]	q9
8	q7	ε	q8
9	q0	ε	q6
10	q9	ε	q1
11	q10	:	q11
12	q1	ε	q10
13	q12	[0-5]	q13
14	q14	[0-9]	q15
15	q13	ε	q14
16	q11	ε	q12

Referências

Referências Bibliográficas

- [1] SOUZA, Daniel Leal. *Atividade Prática AV1: Implementação de Expressões Regulares e AFNε*. Material da disciplina de Linguagens Formais e Autômatos, 2026.
- [2] *Guia de Sintaxe para Apresentação das Expressões Regulares*. Material de apoio da disciplina de Linguagens Formais e Autômatos, 2026.
- [3] *MiniLang — Guia Base e Especificação do Projeto*. Versão vigente do projeto, 2026.
- [4] PYTHON SOFTWARE FOUNDATION. *Regular expression operations*. Documentação do módulo `re`. Disponível em: <https://docs.python.org/3/library/re.html>. Acesso em: 30 set. 2026.